

A 15-MINUTE TALK FOR RUBY DEVELOPERS WHO SHIP WITH CODING AGENTS

# What belongs in your PostgreSQL SKILLS.md file

KONSTANTIN GREDESKOUL, KIG.RE

FIRST, A SHOW OF HANDS

**But please allow me to ask a few questions of you first...**

FIRST, A SHOW OF HANDS

But please allow me to ask a few questions of you first...

Raise your hand if, assuming you are a software engineer, you code regularly with an AI as an assistant? **But you write most of the code...**

FIRST, A SHOW OF HANDS

But please allow me to ask a few questions of you first...

Raise your hand if you barely write any code any more. You write specs, the AI takes over, and **maybe you do manual reviews.**

FIRST, A SHOW OF HANDS

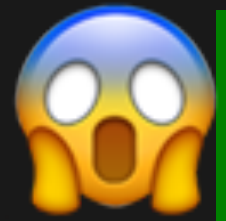
But please allow me to ask a few questions of you first...

Now, raise your hand if you barely write any code any more.

You write specs, the AI workflow with a harness over, **AI does reviews, merges and deploys.**

FIRST, A SHOW OF HANDS

**But please allow me to ask a few questions of you first...**



**Now, a trick question.**

FIRST, A SHOW OF HANDS

But please allow me to ask a few questions of you first...

Have you personally (or maybe someone else at your work) read every single **SKILL.md**, plugin, command, script your AI environment contains?

FIRST, A SHOW OF HANDS

But please allow me to ask a few questions of you first...

Who is picky about what lands in

~/ .claude/skills or ~/ .agents/skills ?

FIRST, A SHOW OF HANDS

**But please allow me to ask a few questions of you first...**

**Anyone has written a skill, command or plugin they use every week?**

# Which database are you building?

## PG-lax

A game or a social app. A lost row costs little.  
Speed matters more than strictness.

## PG-traditional

An online store, or an app with a lot of user data.  
Integrity saves you support tickets.

## PG-strict

Money, taxes, health records. A mistake costs real money, and auditors will ask.

## PG-analytics

A data warehouse. Few users, long queries, batch imports, materialized views.

Your agent cannot guess this from your table names. **You have to tell it.**

# The answer changes the rules

|                        | PG-lax                            | PG-traditional         | PG-strict                             | PG-analytics              |
|------------------------|-----------------------------------|------------------------|---------------------------------------|---------------------------|
| Rough throughput       | 2k to 10k tx/s                    | 500 to 5k tx/s         | 100 to 1k tx/s                        | Under 10 queries/s        |
| Deletes                | Physical                          | Decide per table       | Logical only, <code>deleted_at</code> | Rare                      |
| <code>ON DELETE</code> | <code>CASCADE</code>              | Stated on every key    | Custom, sets <code>deleted_at</code>  | Rarely matters            |
| Row locking            | <code>lock_version</code> is fine | Where money moves      | <code>FOR UPDATE</code> by default    | Few writers               |
| Query mix              | Many small reads                  | Small reads and writes | Small reads and writes                | A few heavy, long queries |

←!— AGENTS.md —→  
Database class: PG-strict. Logical deletes only. Ask before any physical DELETE.

Skip this line and your agent adds ON DELETE CASCADE to a ledger.

## WHY WRITE IT DOWN

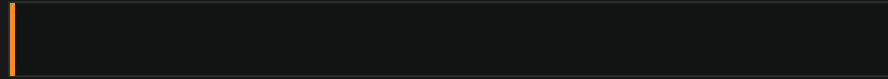
# Your agent writes code faster than you can review it

Code written



~500x

Code reviewed



1x

So I moved my judgment to the place the agent reads before it writes the first line: a skill file.

When the agent adds `lock_timeout` without being asked, the file did its job.

## WRITE YOUR OWN

A skill file changes the code your agent writes. Read someone else's file as carefully as a new gem with production access.

Better, write yours from incidents you lived through. You know things a downloaded file does not.

# A short trigger file, and a long reference file

## THE YAML DESCRIPTION

### When to load

Lists the words that should load the skill: `migration`, `add_index`, `uuidv7`, `deleted_at`, `ON DELETE`.

## SKILL.MD, ABOUT 80 LINES

### What never to miss

Step zero, then eight rules that fail silently. Then it tells the agent to read the reference.

## REFERENCES/PRACTICES.MD

### Why, in detail

The explanations and the measurements. The agent reads it only when it designs schema.

The description says **when** to load the file. If it summarizes the rules, the agent skips the file.

## RULE 1: MIGRATIONS

# A migration can take your site down before it runs

**SELECT**

long report, holds a lock

**ALTER TABLE**

waits for an exclusive lock

**SELECT**

waits

**SELECT**

waits

**SELECT**

waits

**SELECT**

waits

**SELECT**

waits

PostgreSQL grants locks in order of arrival. Every **SELECT** that arrives after your **ALTER** waits behind it.

**strong\_migrations** checks the SQL. It does not check the wait.

```
class AddCurrencyToInvoices < ActiveRecord::Migration[8.0]
  def change
    safety_assured do
      execute "SET lock_timeout = '3s'"
      add_column :invoices, :currency, :string
    end
  end
end
```

Give up after three seconds and retry. Build every index **concurrently**, alone in its own migration.

# Add NOT NULL in three migrations, not one

To enforce that `invoices.currency` is never null, `change_column_null` takes the strongest lock and reads all 200 million rows. The table is unavailable until it finishes.

1

ADD THE RULE, LOCK FOR AN INSTANT

`CHECK (currency IS NOT NULL) NOT VALID` guards every new row and reads none of the old ones.

2

PROVE IT, IN ITS OWN MIGRATION

`VALIDATE CONSTRAINT` reads all 200 million rows under a weaker lock. Your app keeps reading and writing.

3

NOW SET NOT NULL

PostgreSQL 12 and later accept the validated check as the proof, skip the scan, and flip a flag.

Three short locks instead of one long one, and the same column in the end.

# Four rules that fit on one line each

## schema\_format = :sql

`schema.rb` silently drops partial indexes, expression indexes, exclusion constraints, generated columns, and extensions.

## timestampz

`timestamp` stores no zone, so the same value means a different instant to every reader, and daylight saving silently moves it. `timestampz` stores one absolute instant and renders it in the session's zone. Rails still picks the wrong one: set `datetime_type = :timestampz`.

## Foreign keys, with ON DELETE

`CASCADE` deletes the children, `RESTRICT` refuses, `SET NULL` orphans them, and `NO ACTION` is the default nobody chose. PG-lax takes `CASCADE`. PG-strict takes `RESTRICT` and propagates `deleted_at` in code, because a ledger row outlives its parent.

## Vendor data in its own schema

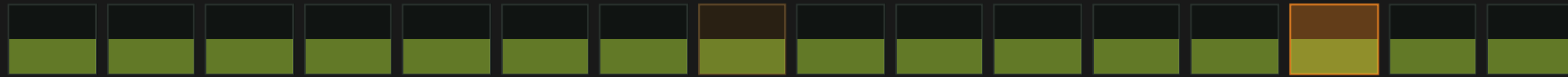
Stripe tables go in `stripe.*`, Plaid tables in `plaid.*`. Never in `public`.

## RULE 4: PRIMARY KEYS

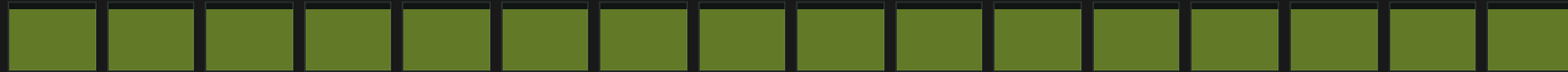
# Use uuidv7 keys, and know what they reveal

```
create_table :invoices, id: :uuid,  
  default: → { "uuidv7()" } do |t|  
  t.timestamps  
end
```

uuidv4: every insert touches a random, half-empty page



uuidv7: inserts fill full pages at the end



### WHAT YOU GET INSTEAD WITH SERIAL KEYS

Rails' old default, `integer`, stops at 2.1 billion. `bigserial` never runs out, and both are fast and compact. Both also publish your business: `/users/10000` tells a competitor how many customers you have, tells the customer they are number ten thousand, and invites anyone to walk `/invoices/1`, `/invoices/2` and see what they can read.

### THE UUIDV7 TRADEOFF

A uuidv7 shows when its row was created. With enough IDs from your URLs, anyone can estimate how fast you grow.

If that matters, show a uuidv4 outside and keep the uuidv7 inside. The extra 8 bytes over `bigint` do not matter. The insert speed does.

# Soft deletes need partial indexes. The index has a cost.

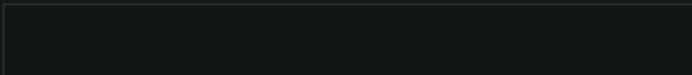
A PG-strict application never physically deletes a record. It sets `deleted_at`, so nearly every query then says `WHERE deleted_at IS NULL`, and a partial index on that condition covers only live rows and stays small.

But a soft delete is still an `UPDATE`, and every `UPDATE` writes a new row version and leaves the old one dead for vacuum.

A HOT update is the cheap kind: the new version fits on the same page and no index is touched. It only qualifies when the update changes no indexed column, and a partial index's `WHERE` clause counts as indexed.

I SOFT-DELETED 200 ROWS TWICE ON POSTGRESQL 18 AND COUNTED THE CHEAP UPDATES

No index on `deleted_at`  174/200

With the partial index  0/200

Add the index and every soft delete costs you index writes. Both runs still left 200 dead rows for vacuum, so keep the index on PG-strict: the reads matter more.

# Order index columns: equality, then range, then sort

```
WHERE account_id = ?  
ORDER BY created_at DESC  
LIMIT 20
```

**SERVES IT** (account\_id, created\_at)

**USELESS** (created\_at, account\_id)

Selectivity only breaks a tie. The shape of the query decides.

RAILS LEAVES AN EXTRA INDEX BEHIND

`add_reference` creates an index on (account\_id).

Three sprints later, someone adds (account\_id, created\_at). Now the first index is redundant, and nobody drops it.

Find the ones nobody uses: `pg_stat_user_indexes` where `idx_scan = 0`.

# A foreign key points at one table. Pick accordingly.

## ONE TABLE, MANY CLASSES

### STI

A real key can point at `carts.id`. It just cannot say **which** subclass, so Postgres accepts a `smoothies.banana_id` that points at a strawberry. Add a `CHECK` for that.

**The downside:** one wide table shared by every subclass, so any column one class needs is nullable for all of them.

## RAILS 6.1 AND LATER

### Delegated types

Each class gets its own table, and a join row points at whichever one applies. That key is real and enforced, and each table can require its own columns.

**The downside:** more tables, an extra join on every read, and Rails wiring the next person has to learn.

## THE ONE THAT GIVES UP THE KEY

### Polymorphic

`edible_id` points at whatever `edible_type` names on that row, and no foreign key can express that. You trade enforcement for convenience.

**The downside:** nothing stops an orphan, nothing cascades, and the database cannot tell you when it happened.

Store `banana` in the type column, never `Fruit::Banana`, or renaming a module becomes a data migration. Index `(edible_id, edible_type DESC)`.

# Everything else goes in the reference file

## Locking

`READ COMMITTED` gives each statement a new snapshot. Use `FOR UPDATE` for read-then-write.

## Money

`bigint` cents plus a `char(3)` currency. Never `float`.

## Timeouts

`statement_timeout` per role: 30 seconds for web, longer for jobs.

## Pooling

pgBouncer in transaction mode. About 16 to 20 server connections on 8 cores.

## N+1 queries

`strict_loading` raises in development instead of running 400 queries in production.

## Wraparound

An open snapshot blocks freezing. Turning autovacuum off does not.

## YOUR TURN

# Write your own file this week

- 1 Classify your database. Write the class in AGENTS.md.
- 2 Write a description that lists the words that should load the file.
- 3 Put the rules that fail silently in SKILL.md. Keep it under 100 lines.
- 4 Move the explanations and measurements to a reference file.
- 5 Add a rule each time an incident teaches you one.

**Then run the unused-index query on production. I dare you.**

BONUS

# Priceless Tangent

## How I run my Software Team

Watch for [dry-cli.tools](#), coming  
online soon.

INSTALLER

### **agentilda-ai-setup**

Installs skills, plugins, commands, and hooks for any agent. Take only the parts you want, from any repo.

[github.com/kigster/agentilda-ai-setup](https://github.com/kigster/agentilda-ai-setup)

RUBY GEM, INSTALLS THE  
`tilda` CLI

### **agentilda**

Does most of the work. It writes and refines a spec, turns it into a plan, builds it, opens the PR, and loops on review until it is approved.

`tilda install skills` teaches  
your agent to use it.

[github.com/kigster/agentilda](https://github.com/kigster/agentilda)

RUBY GEM, INSTALLS THE `alo`  
CLI

### **agent-lock**

Lets concurrent agents lock part of the tree to themselves. It uses local Redis when it can, and the file system when it cannot.

[github.com/kigster/agent-lock](https://github.com/kigster/agent-lock)

# Thanks!

## Was this helpful?

## What could I have done better?

Download my PostgreSQL skill tomorrow from [kig.re/skills](https://kig.re/skills)

I am for hire soon. Talk to me: (415) 265 1054



CONDENSING TWENTY YEARS OF WISDOM IN ONE MARKDOWN, ON KIG.RE